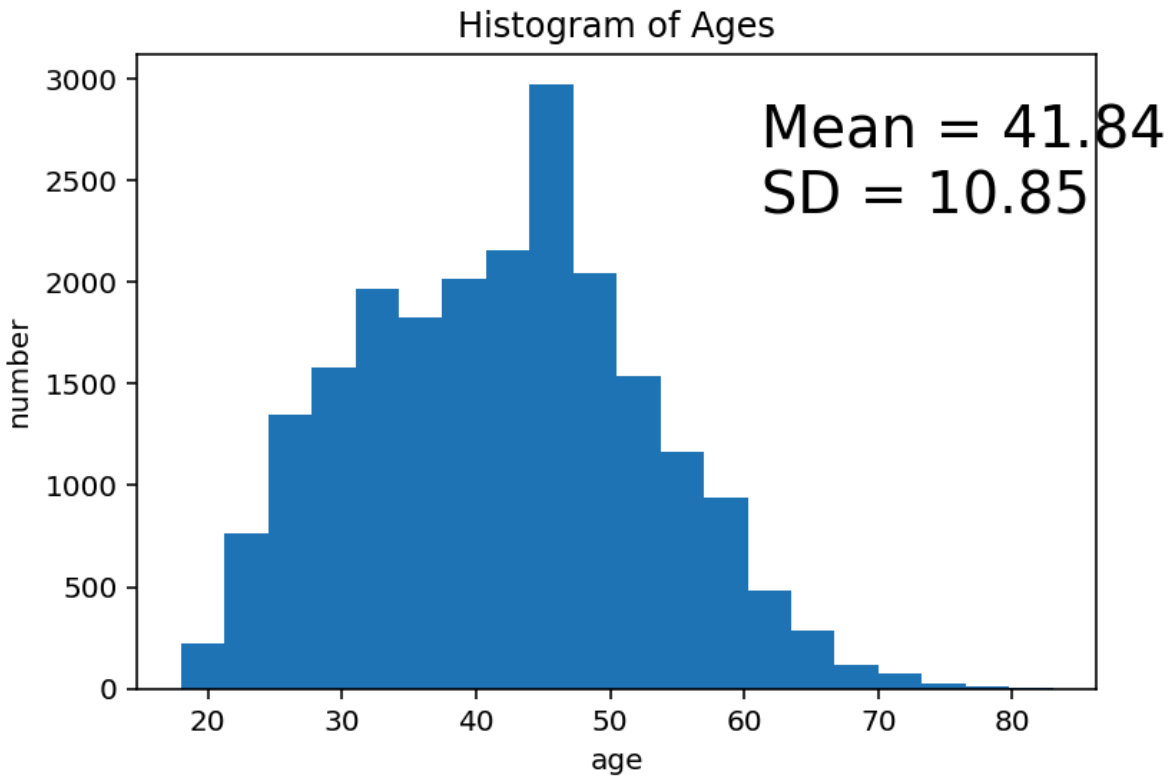
Question: If the shape of a distribution is not normal, what can we infer about its mean?

- The mean still exists and can be calculated. The shape doesn't determine if the mean is valid. Although if the distribution is too skewed the mean probably won't show the usual value as good as the median would. But no matter the shape mean is still the average.

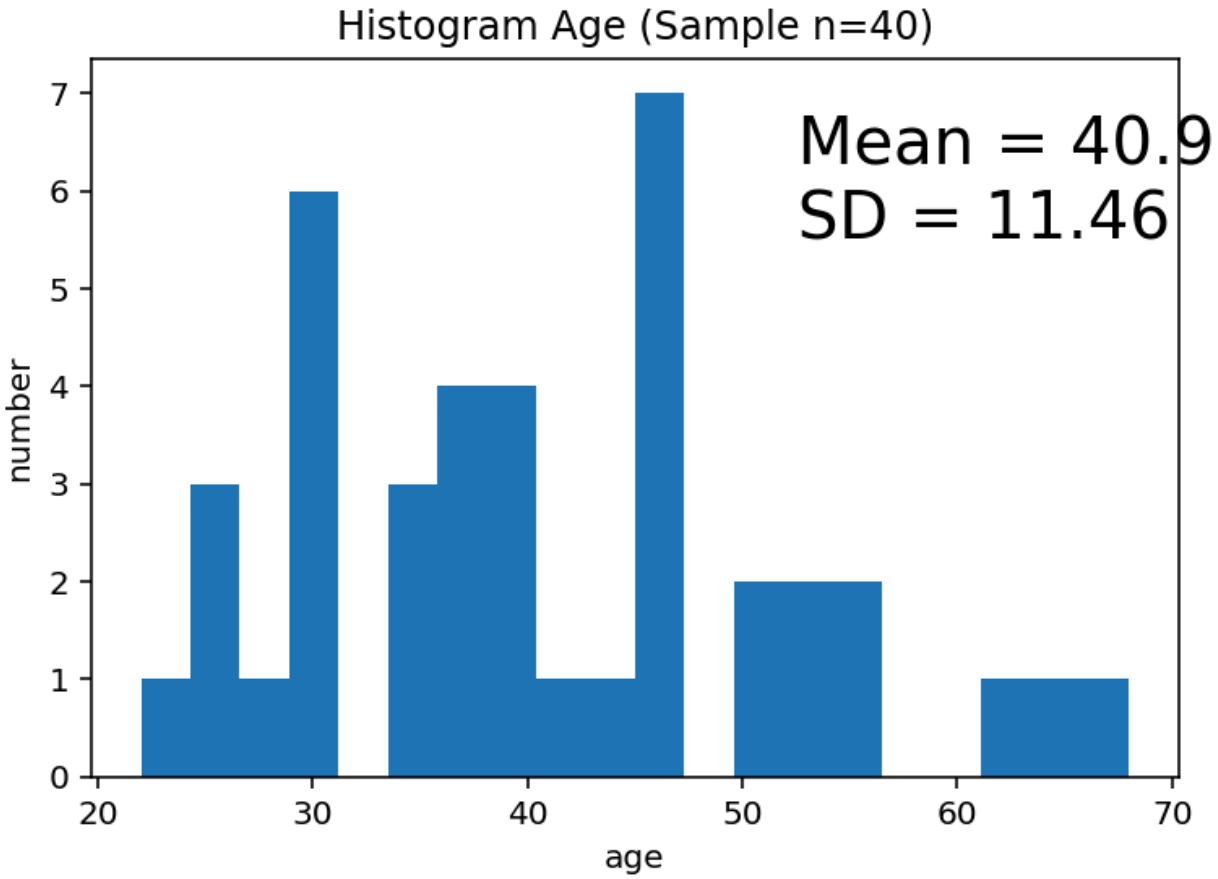
1) The data structure returned is a Dictionary containing a list
2) To pull out "age" from this data structure we use the key "age". So I'm saving it into a variable ages = bmData["age"]
3) This looks roughly like a normal distribution, though it does look like it's slightly skewed since there are some older runners. Overall it is close to bell-shape.
   Mean = 41.84
   SD = 10.85



4) The random sample size 40
   - Mean = 40.9
   - SD = 11.46
   - The sample of 40 isn't as smooth as the population. It looks more uneven and random looking because the sample size is small.
   - Since we're only taking 40 people, the mean should be close to the population but not the same.

- Differences:
  population mean = 41.84, sample mean (40) = 40.9, difference = 0.94
  population SD = 10.85, sample SD (40) = 11.46, difference = 0.61

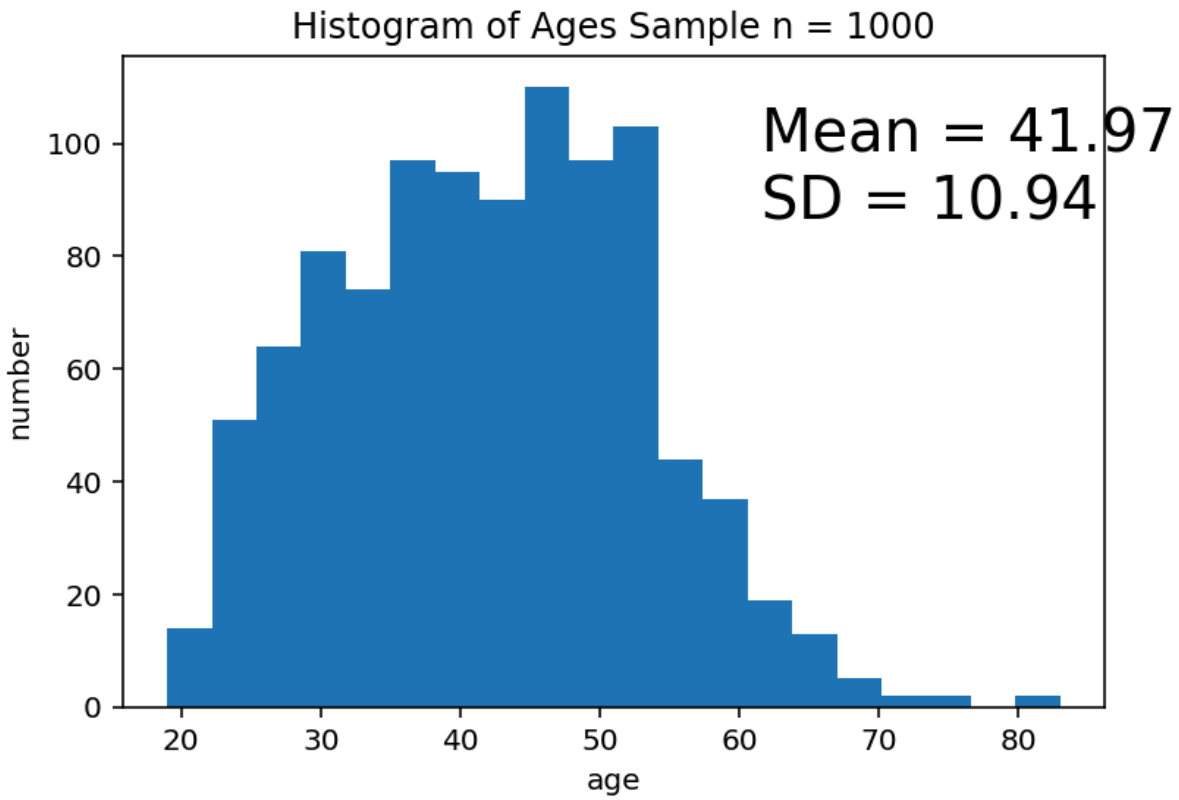
- The sample values are close but not exactly the same.



5) The random sample of size 1000
   - Mean = 41.97, SD = 10.94
   - This histogram looks much more similar to the population than the sample of 40. It is smoother and more centered.
   - Differences:
     Population mean = 41.84
     Sample mean (1000) = 41.97
     Difference = 0.13

     Population SD = 10.85
     Sample SD (1000) = 10.94
     Difference = 0.09

     These differences are much smaller than for the sample of 40. This supports the Law of Large Numbers. As the sample size increases, the sample mean gets closer to the population mean.



6) What happens as N increases
The histogram becomes smoother. The center gets closer to the population mean and the spread becomes closer to the population spread.

Part 2:

For 1 die: The distribution looks roughly uniform. It is not normal since the outcomes are spread fairly evenly.

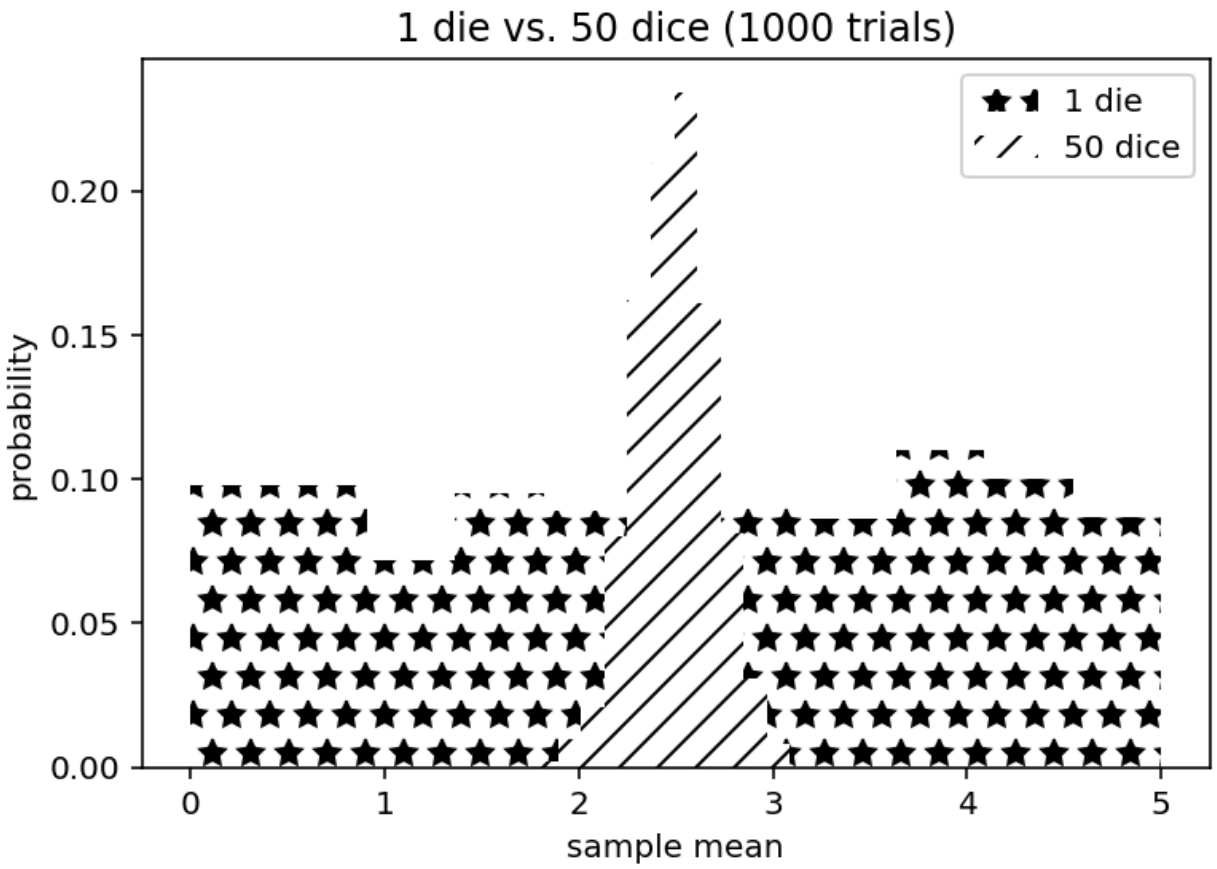
For 50 dice (mean of 50 rolls): The distribution looks much more bell-shaped around the center. It is approximately normal.

This demonstrates the Central Limit Theorem. Even though a single die is not normally distributed, the distribution of the sample means becomes approximately normal when the sample size is large.

Means: The mean of the single rolls (2.4424) and the mean of the 50-roll averages (2.4923) are close. This shows that the expected value of the sample mean equals the population mean.

Standard deviations: The standard deviation of the single rolls is 1.4446, but the standard deviation of the 50-roll means is 0.2040. The distribution of the sample means has much smaller spread. This matches the formula SE=σ/√n. As n increases, the spread decreases.

The standard deviation of the distribution of the 50-roll means (≈ 0.204) is the standard error of the mean.



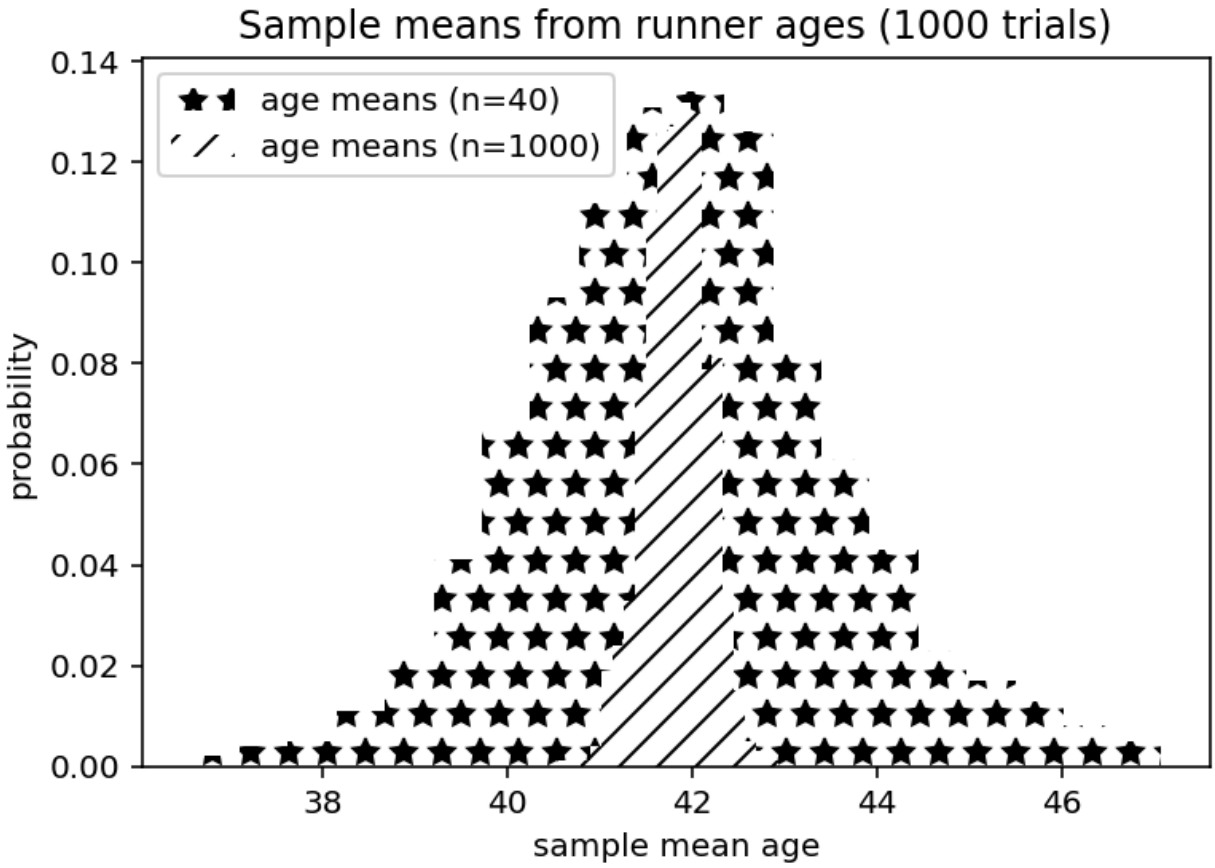
Sampling from Ages:
Same:

- Just like the dice example, the distribution of sample means becomes more normal-looking as sample size increases.
- The center of the sampling distribution stays close to the population mean age (around ~42).
- The spread (standard error) gets smaller when the sample size increases.

Different:

- The ages data is a real dataset and is not perfectly symmetric, so small-sample means may show more unevenness than dice.
- Ages might have slight skew and outliers, so the original population shape is different, but the means still becomes normal-ish

ageMean40: 41.736974999999966 | ageSD40: 1.7064118980893621
ageMean1000: 41.84832300000008 | ageSD1000: 0.3357752010745534



```
import math


def mean(data):
    total = 0.0
    n = len(data)
    for x in data:
        total += x
    return total / n

def median(data):
    s = sorted(data)
    n = len(s)
    mid = n // 2
    if n % 2 == 1:
        return s[mid]
    else:
        return (s[mid - 1] + s[mid]) / 2.0

def mode(data):
    counts = {}
    for x in data:
        if x in counts:
            counts[x] += 1
        else:
            counts[x] = 1

    max_count = 0
    for x in counts:
        if counts[x] > max_count:
            max_count = counts[x]

    modes = []
    for x in counts:
        if counts[x] == max_count:
            modes.append(x)

    return sorted(modes), max_count

def min_value(data):
    m = data[0]
```

```
    for x in data:
        if x < m:
            m = x
    return m

def max_value(data):
    m = data[0]
    for x in data:
        if x > m:
            m = x
    return m

def data_range(data):
    return max_value(data) - min_value(data)

def sum_squared_deviations_definition(data):
    m = mean(data)
    total = 0.0
    for x in data:
        total += (x - m) ** 2
    return total

def sum_squared_deviations_computational(data):
    n = len(data)
    sum_x = 0.0
    sum_x2 = 0.0
    for x in data:
        sum_x += x
        sum_x2 += x * x
    return sum_x2 - (sum_x * sum_x) / n

def variance_population(data):
    ssd = sum_squared_deviations_definition(data)
    n = len(data)
    return ssd / n

def variance_sample(data):
    ssd = sum_squared_deviations_definition(data)
    n = len(data)
    return ssd / (n - 1)

def std_population(data):
    return math.sqrt(variance_population(data))

def std_sample(data):
    return math.sqrt(variance_sample(data))

def describe(data, use_sample=True):
    n = len(data)
    m = mean(data)
    med = median(data)
```

```
    modes, freq = mode(data)

    if use_sample:
        var = variance_sample(data)
        sd = std_sample(data)
        which = "(Sample)"
    else:
        var = variance_population(data)
        sd = std_population(data)
        which = "(Ppopulation)"

    mn = min_value(data)
    mx = max_value(data)
    r = mx - mn

    print(which )
    print("N:", n)
    print("Mean:", m)
    print("Median:", med)
    print("Mode(s):", modes, " (frequency:", freq, ")")
    print("Min:", mn)
    print("Max:", mx)
    print("Range:", r)
    print("Variance:", var)
    print("Std Dev:", sd)
    print("-------------------------------------------")

students = [18,18,18,19,19,19,19,19,20,22,22,22,23,23,24,25,26,26,29]

describe(students, use_sample=True)
describe(students, use_sample=False)

print("SSD (definition):", sum_squared_deviations_definition(students))
print("SSD (computational):", sum_squared_deviations_computational(students))
```